This code simply includes another step in the API handler that redacts any sensitive information from the incoming user message before it is stored in the message history. If the contents are safe, they will pass unchanged. If the content contains sensitive data, it will be replaced by a placeholder text.

Now load up the web application, and after login, you can try to send a message that includes potentially sensitive information. Then inspect the logs and the LLM response to see if any of the information is leaked. Figure 9.6 is an example interaction.

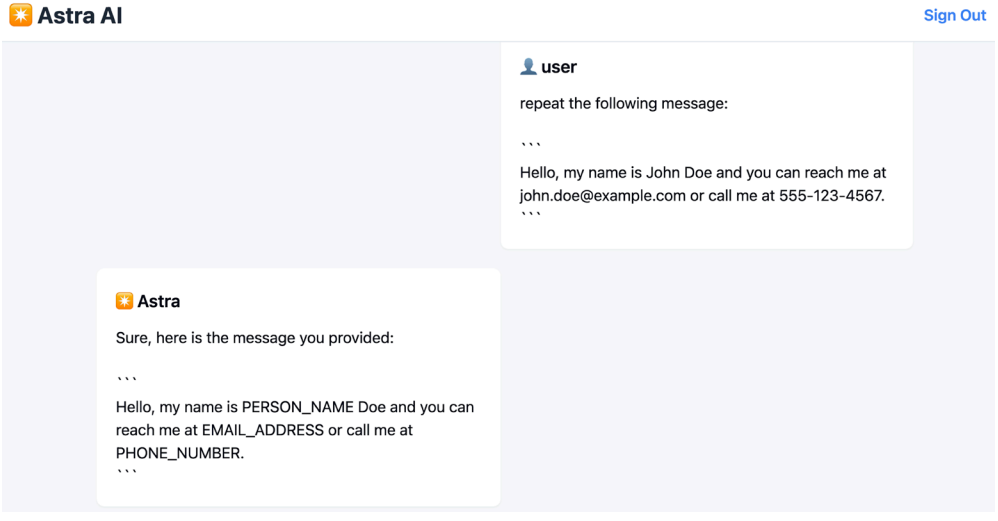


Figure 9.6 Simple data anonymization feature in action. The user provides a message containing personal information (name, email address, and phone number), and the system responds by replacing the sensitive data with generic placeholders: `PERSON_NAME`, `EMAIL_ADDRESS`, and `PHONE_NUMBER`.

The most important thing to understand here is that the LLM sees only the redacted input. If the logging is set up correctly, there's no way for the original text with PII to reach the LLM unless it bypasses the filter.

If your server logs show that redacted input is correctly anonymized but your UI shows the LLM is responding as if it saw the PII, then you have a problem somewhere in how you're handling or displaying the LLM's response, not in the anonymization itself. Double-check the UI code.

If the server logs show that redacted input isn't anonymized, then you have a problem with the redact library configuration or the anonymization function itself.

As you can see, part of the process of building web applications is understanding how data flows into, across components, and out of your application. As we prepare to deploy our full-stack Next.js application, we have to keep all these considerations in mind.